

T.C. KARADENİZ TEKNİK ÜNİVERSİTESİ

OF/TEKNOLOJİ FAKÜLTESİ

YAZILIM MÜHENDİSLİĞİ BÖLÜMÜ

CATLARY KÜTÜPHANE YÖNETİM SİSTEMİ Proje Raporu

Ders Adı: Veri Tabanı-Hakan Aydın

Öğrenci Ad-soyad: Melisa Taşkara(436581)

Github hesabı: <https://github.com/melisa-6>

Teslim Tarihi: 29 / 12 / 2025

VIDEO TANITIMI: Projenin tanıtımı amacıyla demo videosu hazırlanmış ve YouTube platformunda linki bulunan kanala yüklenmiştir: www.youtube.com/@Catlaryy

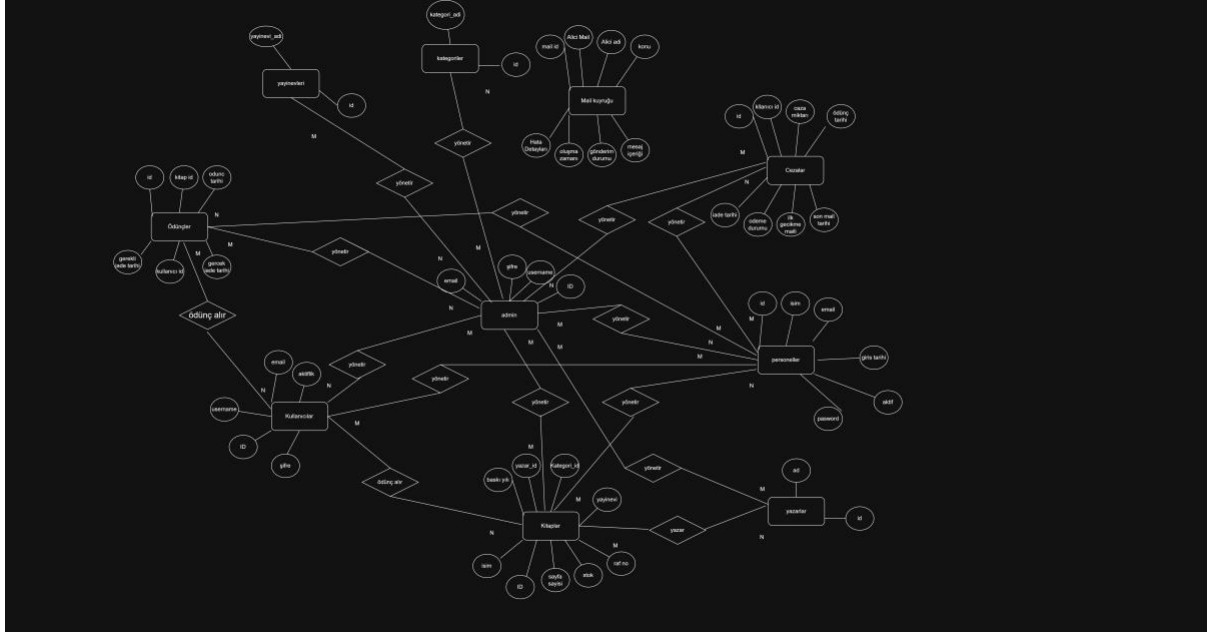
PROJENİN AMACI VE KAPSAMI

Catlary Kütüphane Yönetim Sistemi projesinin temel amacı, bir kütüphanede bulunan kitapların, kullanıcıların ve ödünç alma işlemlerinin dijital ortamda yönetilebildiği bir sistem geliştirmektir; bu kapsamda kütüphane yönetim süreçlerinin otomatikleştirilmesi, kitap, kullanıcı, yazar ve kategori yönetiminin sağlanması, ödünç alma ve iade işlemlerinin kontrol altına alınması, geç iade durumlarında otomatik ceza hesaplanması ve veritabanı ile entegre çalışan REST tabanlı API'lerin geliştirilmesi hedeflenmiştir. Proje kapsamında JWT tabanlı kullanıcı girişi ve kimlik doğrulama, kitap arama ve listeleme, kitap ödünç alma ve iade işlemleri, geç iade durumlarında ceza hesaplama ve kayıt altına alma, kullanıcılar için CRUD işlemleri, API testleri ve temel kullanıcı arayüzlerinin geliştirilmesi yer almaktadır. Projede Backend tarafında Flask (Python), veri tabanı olarak MySQL, frontend tarafında HTML, CSS ve JavaScript teknolojileri kullanılmış, kimlik doğrulama JWT ile sağlanmış ve API testleri Postman üzerinden gerçekleştirilmiştir. Sistem mimarisi olarak katmanlı mimari benimsenmiş olup bu yapı sayesinde kodun okunabilirliği, sürdürülebilirliği ve yönetilebilirliği artırılmış; mimari Entity katmanı (veri tabanı tablolarını temsil eden sınıflar), Repository katmanı (veri tabanı işlemleri), Service katmanı (iş kuralları) ve Controller katmanı (API uç noktaları) olmak üzere dört ana katmandan oluşturulmuştur.

VERİTABANI TASARIMI

Projede ilişkisel veritabanı modeli kullanılmıştır. Sistemde yer alan temel varlıklar Admin, Personel, Kullanıcı, Kitap, Yazar, Kategori, Ödünç, Ceza ve Yayınevi'dir.

Bu varlıklar arasındaki ilişkiler aşağıdaki ER diyagramı ile gösterilmiştir.



FRONTEND TASARIMI

Frontend tarafında kullanıcıların sistemle etkileşime geçebileceği temel arayüzler geliştirilmiştir. Anasayfa ve giriş sayfaları kapsamında **anasayfa.html** dosyası sistemin genel tanıtımının yapıldığı ve kullanıcıların giriş yönlendirmelerinin bulunduğu ana ekran olarak tasarlanmış, **kullanici.html** giriş yapan kullanıcıların temel işlemleri gerçekleştirdiği kullanıcı paneli olarak kullanılmış, **personel.html** giriş yapan personellerin rollerine uygun işlemleri gerçekleştirdiği personel paneli, **admin.html** ise admin yetkisine sahip kullanıcıların yönetim işlemlerini gerçekleştirdiği admin paneli olarak geliştirilmiştir. Kitap, yazar ve kategori yönetimi için **kitaplar.html** kitap arama, listeleme, silme, düzenleme ve stok görüntüleme işlemlerini, **yazarlar.html** yazar ekleme, silme ve listeleme işlemlerini, **kategoriler.html** kategori ekleme, silme ve listeleme işlemlerini, **yayinevleri.html** ise yayınevi ekleme, silme ve listeleme işlemlerini sağlamaktadır. Ödünç alma ve iade işlemleri kapsamında **oduncver.html** kitap ödünç verme işlemleri için, **odunc_gecmisim.html** kullanıcının kendi ödünç geçmişini görüntülemesi için, **tum_odunc_gecmisi.html** ise admin ve personelin tüm ödünç kayıtlarını görüntülemesi için tasarlanmıştır. Ceza ve borç işlemlerinde **borcode.html** kullanıcının ceza borçlarını ödeyebildiği ekranı, **cezalarim.html** kullanıcıya ait gecikme cezalarının listelendiği sayfayı, **tum_cezalar.html** ise admin ve personelin tüm ceza kayıtlarını görüntülediği ekranı sunmaktadır. Admin arayüzü kapsamında **admin.html** admin ana sayfası olarak yönetim işlemlerine erişimi sağlarken, **adminler.html** sistemde tanımlı admin kullanıcıların listelendiği ekranı, **kullanicilar.html** kullanıcı ekleme, silme ve durum yönetimi işlemlerini, **personel.html** personel ekleme ve yönetim işlemlerini, **personel_listesi.html** ise kayıtlı personellerin listelendiği sayfayı içerecek şekilde geliştirilmiştir.

TEST SÜRECİ

Geliştirilen API'ler Postman aracılığıyla test edilmiştir. Başarılı ve hatalı senaryolar değerlendirilmiştir.

Örnek senaryo(GET isteği);

Postmanden Endpoint gelen istek Controller katmanına yönlendirir. Sonra sırasıyla service ve repository katmanlarına yönlendirilir.

```
@yazar_bp.route("/yazarlar", methods=["GET"])
@login_required # Giriş zorunluluğu.
def yazarlar_getir_route():
    # İstemcinin JSON yanıtı isteyip istemediğini kontrol eder
    is_api_request = request.accept_mimetypes.best == "application/json" or request.is_json

    # Controller'dan yazarları çeker
    yazarlar = yazarlar_controller.tum_yazarlari_getir_controller()

    basarili_mi = True
    mesaj = "Yazarlar listelendi"

    # API isteği için JSON dönüşü.
    if is_api_request:
        return jsonify({
            "success": basarili_mi,
            "message": mesaj,
            "yazarlar": yazarlar
        }), 200

    # Web tarayıcısı için HTML dönüşü.
    return render_template(
        "yazarlar.html",
        yazarlar=yazarlar,
        username=getattr(g, "username", "")
    )
```

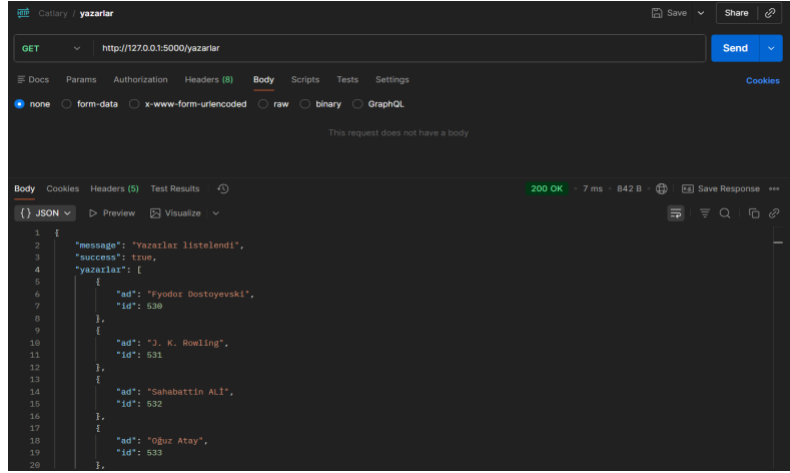
```
def tum_yazarlari_getir_controller(self):
    #uygun service e yonlendirir
    try:
        return self.service.tum_yazarlar()
    except Exception as e:
        print(f"Controller HATA: Yazarlar çekilemedi: {e}")
        return []
```

```
def tum_yazarlar(self):
    #uygun repoya yonlendirir
    return self.repo.tum_yazarlar()
```

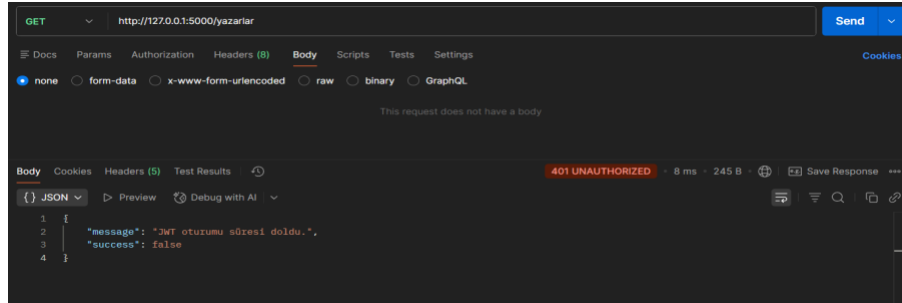
```
def tum_yazarlar(self):
    #yazarlar tablosundan tum yazarlari ceker
    try:
        cursor = self.connection.cursor(dictionary=True)
        #sonuclari dict formatinda dondurmeye icin
        cursor.execute("SELECT * FROM yazarlar")
        sonuc = cursor.fetchall()
        cursor.close()
        return sonuc
    except Exception as e:
        print("Repository HATA:", e)
        return []
```

Postman'den isteğin gönderilmesi:

*Başarılı olma durumu



*Hata verme durumu



Örnek senaryo (Post isteği);

Postmanden Endpointe gelen istek Controller katmanına yönlendirir. Sonra sırasıyla service ve repository katmanlarına yönlendirilir.

```
@yazar_bp.route("/yazarekle", methods=["POST"])
def yazarekle():
    # yazar adini alir
    ad = request.form.get("yazar_ad") or (request.json.get("yazar_ad") if request.is_json else None)

    # controller ile yazarı ekler
    success, msg = yazarlar_controller.yazarekle_controller(ad)

    # Eger istek json formatındaysa json dön.
    if request.is_json:
        return jsonify({
            "success": success,
            "message": msg
        }), (200 if success else 400)

    # Değilse flash mesajı oluştur.
    flash(msg, "success" if success else "danger")

    yazarlar = yazarlar_controller.tum_yazarlari_getir_controller()
    return render_template("yazarlar.html", yazarlar=yazarlar)
```

```
def yazarekle_controller(self, ad):
    mevcut = self.service.yazar_bul(ad)

    #yazar mevcutsa hata verir
    if mevcut:
        return False, "Bu yazar zaten kayıtlı!"

    #mevcut değilse eklemek için ilgili service yönlendirir
    self.service.yazar_ekle(ad)
    return True, "Yazar başarıyla eklendi."
```

```
#uygun repoya yönlendirir
def yazar_bul(self, ad):
    return self.repo.yazar_bul(ad)

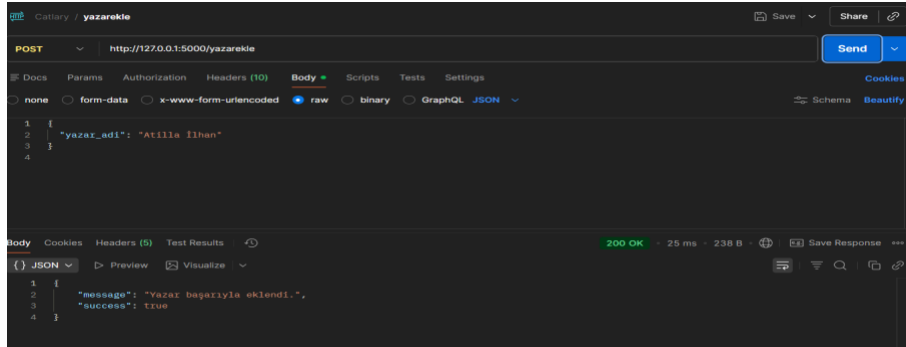
#uygun repoya yönlendirir
def yazar_ekle(self, ad):
    return self.repo.yazar_ekle(ad)
```

```
def yazar_bul(self, ad):
    try:
        cursor = self.connection.cursor(dictionary=True)
        sql = "SELECT * FROM yazarlar WHERE ad = %s"
        cursor.execute(sql, (ad,))
        sonuc = cursor.fetchone()
        cursor.close()
        return sonuc
    except Exception as e:
        print("Yazar Bul Hatası:", e)
        return None

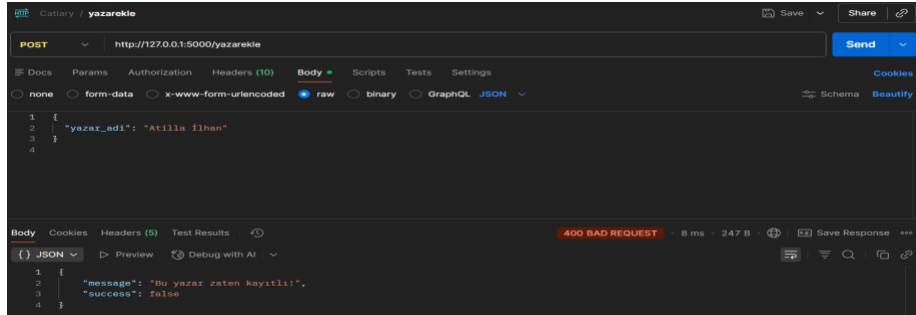
def yazar_ekle(self, yazar_adi):
    try:
        cursor = self.connection.cursor()
        query = "INSERT INTO yazarlar (ad) VALUES (%s)"
        cursor.execute(query, (yazar_adi,))
        self.connection.commit()
        cursor.close()
        return True
    except Exception as e:
        print("Ekleme Hatası:", e)
        return False
```

*Postman'den isteğin gönderilmesi:

*Başarılı olma durumu:

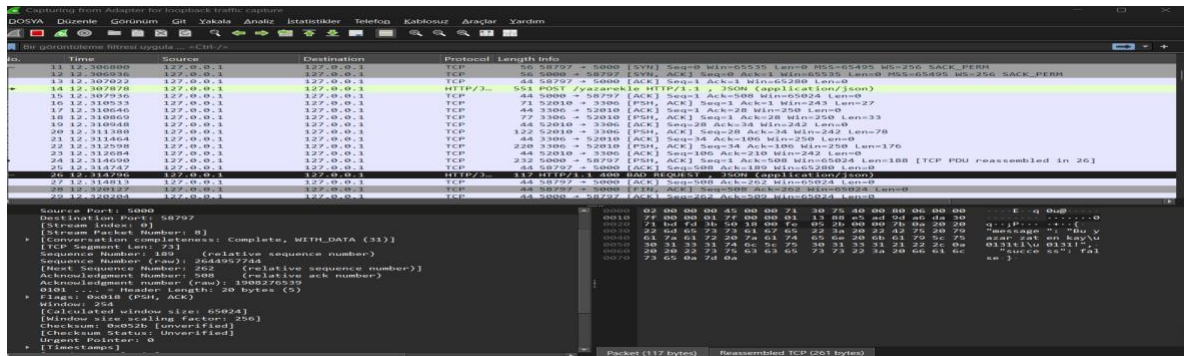


*Hata durumu:



İsteklerin WireShark'ta görüntülenmesi:

*Hata durumu



*Başarılı olma durumu:

